LAPORAN PRAKTIKUM
Sistem Operasi

MODUL 05
(Eksplorasi Proses)

Disusun Oleh:
Ryan Gabriel Togar Simamora (2311104045)
Kelas S1SE-07-02

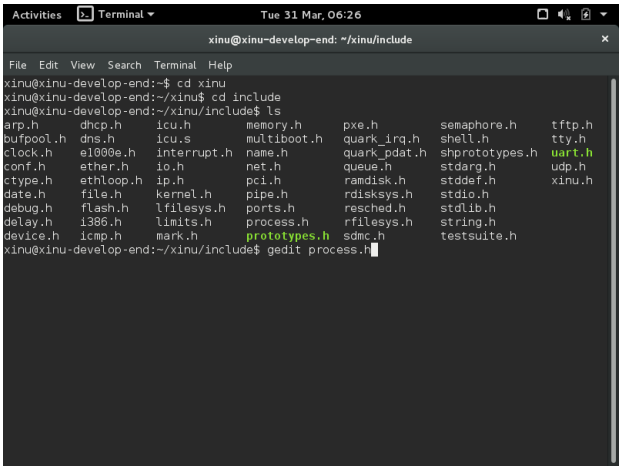
Dosen : Yudha Islami Sulistya, S.Kom., M.Cs

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2025

BAB I
PENDAHULUAN

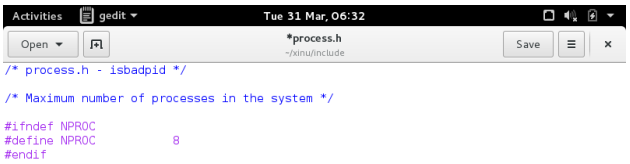
1.1 Dasar Teori

A. Process.h



File process.h merupakan salah satu file header penting dalam sistem operasi Xinu. File ini berisi berbagai definisi yang digunakan untuk mengatur dan mengelola proses di dalam sistem, termasuk jumlah maksimum proses, status proses, dan parameter inisialisasi proses.

1. Jumlah Maksimum Proses



Bagian ini menentukan jumlah maksimum proses yang dapat dijalankan dalam sistem Xinu. Secara default, nilai NPROC adalah 8, artinya hanya delapan proses yang bisa aktif dalam satu waktu, termasuk proses utama dan proses null. Nilai ini bisa diubah sesuai kebutuhan sistem.

2. Konstanta Status Proses

```
/* Process state constants */

#define PR_FREE         0       /* Process table entry is unused        */
#define PR_CURR         1       /* Process is currently running         */
#define PR_READY        2       /* Process is on ready queue            */
#define PR_RECV         3       /* Process waiting for message          */
#define PR_SLEEP        4       /* Process is sleeping                  */
#define PR_SUSP         5       /* Process is suspended                 */
#define PR_WAIT         6       /* Process is on semaphore queue        */
#define PR_RECTIM       7       /* Process is receiving with timeout    */
```

Konstanta-konstanta ini mendefinisikan status dari setiap proses yang ada dalam sistem. Status ini disimpan di tabel proses dan digunakan oleh kernel untuk menentukan tindakan berikutnya terhadap proses tersebut.

3. Definisi Tambahan

```
/* Miscellaneous process definitions */

#define PNMLEN          16      /* Length of process "name"            */
#define NULLPROC        0       /* ID of the null process              */
```

- PNMLEN menentukan panjang maksimum nama proses.
- NULLPROC adalah proses khusus yang selalu berjalan ketika tidak ada proses lain yang aktif.
  Dengan kata lain, NULLPROC memastikan CPU tidak pernah benar-benar idle.

4. Konstanta Inisialisasi Proses

```
/* Process initialization constants */

#define INITSTK         65536   /* Initial process stack size          */
#define INITPRIO        20      /* Initial process priority            */
#define INITRET         userret /* Address to which process returns    */
```

Nilai-nilai di atas digunakan saat proses baru dibuat.
Mereka menentukan ukuran stack yang digunakan proses, prioritas awalnya, dan alamat fungsi yang akan dipanggil ketika proses selesai dieksekusi.

File process.h berperan penting dalam sistem operasi Xinu karena menjadi pusat pengaturan dan definisi yang berhubungan dengan proses.
Mulai dari batas jumlah proses, status proses, hingga parameter inisialisasi, semuanya diatur di sini agar sistem dapat melakukan manajemen proses dengan efisien dan terstruktur.

5. Struktur Data Proses pada Xinu (struct procent)

Selain mendefinisikan konstanta dan batas jumlah proses, file process.h juga berisi struktur utama yang merepresentasikan setiap proses di sistem Xinu, yaitu struct procent.

a. Validasi ID Proses

```
#define isbadpid(x)     ( ((pid32)(x) < 0) || \
                          ((pid32)(x) >= NPROC) || \
                          (proctab[(x)].prstate == PR_FREE))
```

Makro ini digunakan untuk mengecek apakah suatu Process ID (PID) valid atau tidak.

- Jika nilai PID kurang dari 0, melebihi jumlah maksimum proses (NPROC), atau proses tersebut berstatus PR_FREE, maka PID dianggap tidak valid.
- Mekanisme ini membantu mencegah kesalahan akses terhadap entri proses yang tidak ada di tabel proses.

b. Jumlah Device Descriptor

```
/* Number of device descriptors a process can have open */

#define NDESC           5       /* Must be odd to make procent 4N bytes */
```

Setiap proses dapat membuka hingga **5 device descriptor**.
Nilai ini menentukan berapa banyak perangkat (seperti terminal, UART, atau file virtual) yang bisa dihubungkan ke satu proses.
Komentar di kode (Must be odd to make procent 4N bytes) menunjukkan bahwa jumlah ini dipilih agar ukuran struktur tetap sejajar dengan batas memori (kelipatan 4 byte).

c. Struktur Tabel Proses (struct procent)

```
/* Definition of the process table (multiple of 32 bits) */

struct procent {                /* Entry in the process table          */
        uint16  prstate;        /* Process state: PR_CURR, etc.        */
        pri16   prprio;         /* Process priority                    */
        char    *prstkptr;      /* Saved stack pointer                 */
        char    *prstkbase;     /* Base of run time stack              */
        uint32  prstklen;       /* Stack length in bytes               */
        char    prname[PNMLEN]; /* Process name                        */
        sid32   prsem;          /* Semaphore on which process waits    */
        pid32   prparent;       /* ID of the creating process          */
        umsg32  prmsg;          /* Message sent to this process        */
        bool8   prhasmsg;       /* Nonzero iff msg is valid            */
        int16   prdesc[NDESC];  /* Device descriptors for process      */
};
```

Struktur struct procent adalah inti dari manajemen proses dalam Xinu.
Setiap entri di tabel proses (proctab[]) berisi informasi lengkap mengenai status, prioritas, stack, dan perangkat yang digunakan oleh proses. Dengan desain sederhana ini, Xinu dapat melakukan penjadwalan, komunikasi antarproses, serta pengelolaan sumber daya dengan efisien.

d. Struktur Global dan File Implementasi Proses di Xinu

Di bagian akhir file process.h, terdapat beberapa variabel global dan definisi tambahan yang berperan penting dalam pengelolaan proses di sistem Xinu yaitu :

1. Marker Stack (Deteksi Overflow)

```
/* Marker for the top of a process stack (used to help detect overflow) */
#define STACKMAGIC      0x0A0AAAA9
```

Konstanta ini digunakan sebagai penanda (marker) di bagian atas stack proses.
Tujuannya adalah untuk membantu mendeteksi overflow pada stack.
Ketika sistem memeriksa nilai marker ini, jika nilainya berubah, berarti proses telah menggunakan memori stack melebihi batas yang ditentukan indikasi terjadinya stack overflow.

2. Variabel Global untuk Manajemen Proses

```
extern  struct  procent proctab[];
extern  int32   prcount;        /* Currently active processes          */
extern  pid32   currpid;        /* Currently executing process         */
                                C/C++/ObjC Header ▾  Tab Width: 8 ▾   Ln 7, Col 17
```

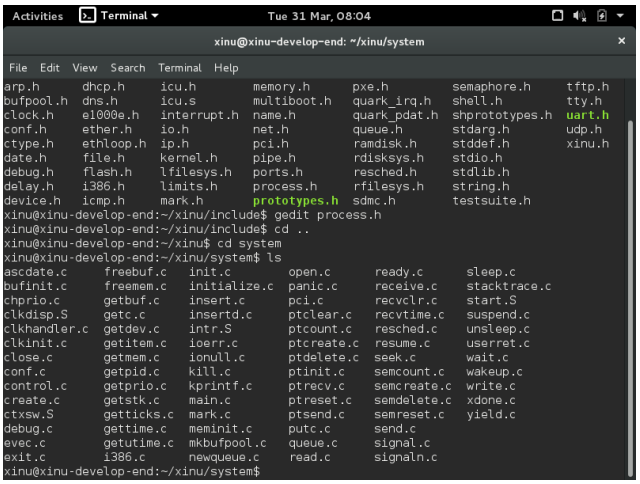
Keterangan:

- proctab[] adalah tabel utama proses yang menyimpan semua data proses aktif di sistem.
- prcount menyimpan jumlah total proses yang sedang aktif.
- currpid menunjuk ke proses yang sedang dieksekusi oleh CPU.

Semua variabel ini bersifat global dan digunakan oleh berbagai fungsi manajemen proses di Xinu (misalnya create(), resume(), sleep(), dan sebagainya).

B. Implementasi Fungsi Proses

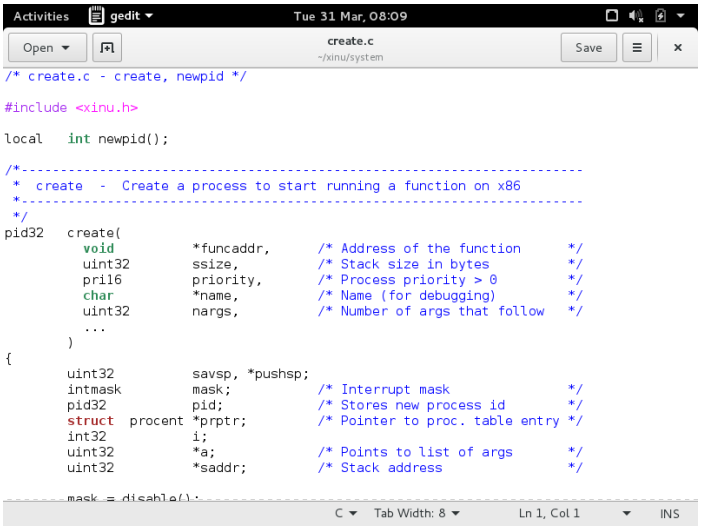
Setelah keluar dari direktori include, kita berpindah ke folder:



Folder ini berisi **file-file sumber C (.c)** yang mengimplementasikan fungsi-fungsi sistem operasi terkait proses, seperti:

- create.cMembuat proses baru
- resume.cMelanjutkan proses yang di-*suspend*
- sleep.cMenidurkan proses untuk jangka waktu tertentu
- ready.cMemasukkan proses ke antrean siap jalan
- resched.cMengatur penjadwalan proses (scheduler)
- kill.cMenghapus atau menghentikan proses
- semcreate.c / semdel.cMengatur semaphore
- send.c / receive.cMengatur komunikasi antarproses
- wait.c / signal.cSinkronisasi antarproses

1. **File:** create.c




```
/* create.c - create, newpid */

#include <xinu.h>

local   int newpid();

/*------------------------------------------------------------------------
 *  create  -  Create a process to start running a function on x86
 *------------------------------------------------------------------------
 */
pid32   create(
          void          *funcaddr,      /* Address of the function      */
          uint32        ssize,          /* Stack size in bytes          */
          pri16         priority,       /* Process priority > 0         */
          char          *name,          /* Name (for debugging)         */
          uint32        nargs,          /* Number of args that follow   */
          ...
        )
{
        uint32          savsp, *pushsp;
        intmask         mask;           /* Interrupt mask               */
        pid32           pid;            /* Stores new process id        */
        struct  procent *prptr;         /* Pointer to proc. table entry */
        int32           i;
        uint32          *a;             /* Points to list of args       */
        uint32          *saddr;         /* Stack address                */

        mask = disable();
```

Header

- #include <xinu.h> → Memasukkan semua definisi dan struktur penting (seperti procent, pid32, dan lain-lain).
- local int newpid(); → Deklarasi fungsi lokal newpid() yang tugasnya mencari ID proses baru yang belum dipakai.

Fungsi utama: create()

```
/*------------------------------------------------------------------------
 *  create  -  Create a process to start running a function on x86
 *------------------------------------------------------------------------
 */
pid32   create(
          void          *funcaddr,      /* Address of the function      */
          uint32        ssize,          /* Stack size in bytes          */
          pri16         priority,       /* Process priority > 0         */
          char          *name,          /* Name (for debugging)         */
          uint32        nargs,          /* Number of args that follow   */
          ...
        )
```

Tujuan:

Membuat proses baru yang siap dijalankan oleh scheduler, tapi belum langsung dijalankan.
Parameter penting:

- funcaddr → pointer ke fungsi yang akan dijalankan oleh proses baru.
- ssize → besar stack dalam byte (harus cukup untuk menyimpan semua argumen dan state).
- priority → menentukan urutan eksekusi (semakin tinggi, semakin diutamakan).
- name → untuk identifikasi/debugging di tabel proses.
- nargs → jumlah argumen tambahan untuk fungsi target.

Variabel internal:

```
{
        uint32          savsp, *pushsp;
        intmask         mask;           /* Interrupt mask               */
        pid32           pid;            /* Stores new process id        */
        struct  procent *prptr;         /* Pointer to proc. table entry */
        int32           i;
        uint32          *a;             /* Points to list of args       */
        uint32          *saddr;         /* Stack address                */
```

Maknanya:

- savsp dan pushsp → menyimpan posisi stack pointer.
- mask → digunakan untuk menonaktifkan interrupt selama pembuatan proses (agar atomik).
- pid → ID proses baru.
- prptr → pointer ke entri di tabel proses (proctab[]).
- a dan saddr → mengatur argumen dan alamat stack.

2. File : **kill.c**

```
/* kill.c - kill */

#include <xinu.h>

/*------------------------------------------------------------------------
 *  kill  -  Kill a process and remove it from the system
 *------------------------------------------------------------------------
 */
syscall kill(
         pid32          pid             /* ID of process to kill        */
        )
{
        intmask mask;                   /* Saved interrupt mask         */
        struct  procent *prptr;         /* Ptr to process's table entry */
        int32   i;                      /* Index into descriptors       */

        mask = disable();
        if (isbadpid(pid) || (pid == NULLPROC)
            || ((prptr = &proctab[pid])->prstate) == PR_FREE) {
                restore(mask);
                return SYSERR;
        }

        if (--prcount <= 1) {           /* Last user process completes  */
                xdone();
        }

        send(prptr->prparent, pid);
```

file kill.c berisi implementasi **fungsi kill()**, yaitu kebalikan dari create().
Kalau create() menambahkan proses ke sistem, kill() menghapusnya dari tabel proses dan melepaskan semua resource yang dipakai.

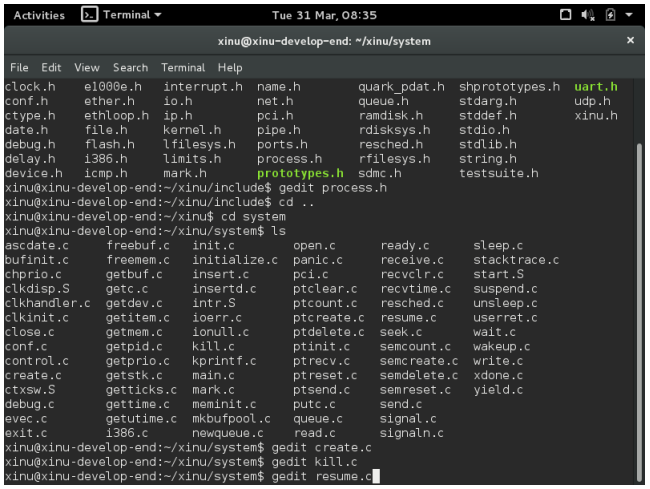
Fungsi kill() **di Xinu**

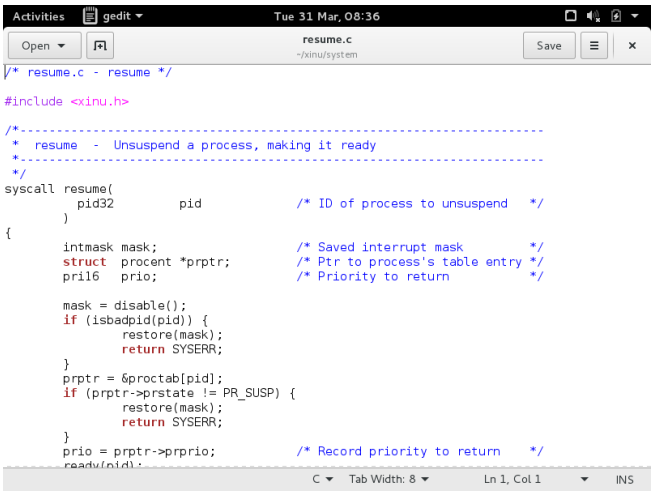
Letak:
/system/kill.c

Tujuan:
Fungsi kill() digunakan untuk menghapus proses dari sistem berdasarkan ID proses (pid). Ketika proses di-*kill*, semua resource (seperti stack dan file descriptor) dibebaskan, dan sistem memperbarui status proses agar slot-nya bisa dipakai lagi oleh proses baru.

Fungsi kill() adalah mekanisme inti untuk **manajemen siklus hidup proses** di Xinu. Proses yang dibuat lewat create() dan diberi PID oleh newpid() akhirnya akan dihapus lewat kill() — inilah rantai penuh dari *process lifecycle management* di kernel Xinu.

3. File : **resume.c**





File resume.c berfungsi untuk mengaktifkan kembali (unsuspend) suatu proses yang sebelumnya disuspensi, sehingga proses tersebut dapat kembali ke status READY dan siap dijalankan oleh scheduler.

Tujuan Praktikum:

1. Mahasiswa memahami Process Control Block (PCB) pada Xinu.
2. Mahasiswa mampu melakukan modifikasi sederhana pada process.

Manfaat Praktikum :

1. Mahasiswa dapat memahami struktur dan fungsi Process Control Block (PCB) pada sistem operasi Xinu, sehingga mengetahui bagaimana sistem mengelola informasi setiap proses.

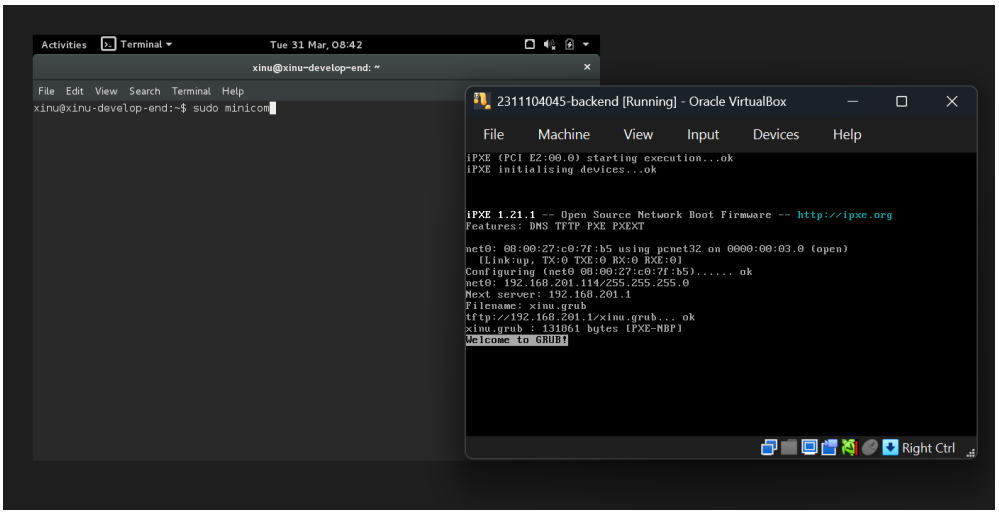
2. Mahasiswa memperoleh kemampuan untuk menganalisis dan memodifikasi proses pada sistem operasi Xinu secara sederhana, yang dapat meningkatkan pemahaman terhadap manajemen proses.

3. Mahasiswa dapat mengembangkan keterampilan praktis dalam memahami cara kerja sistem operasi, khususnya dalam pengelolaan proses dan eksekusinya.

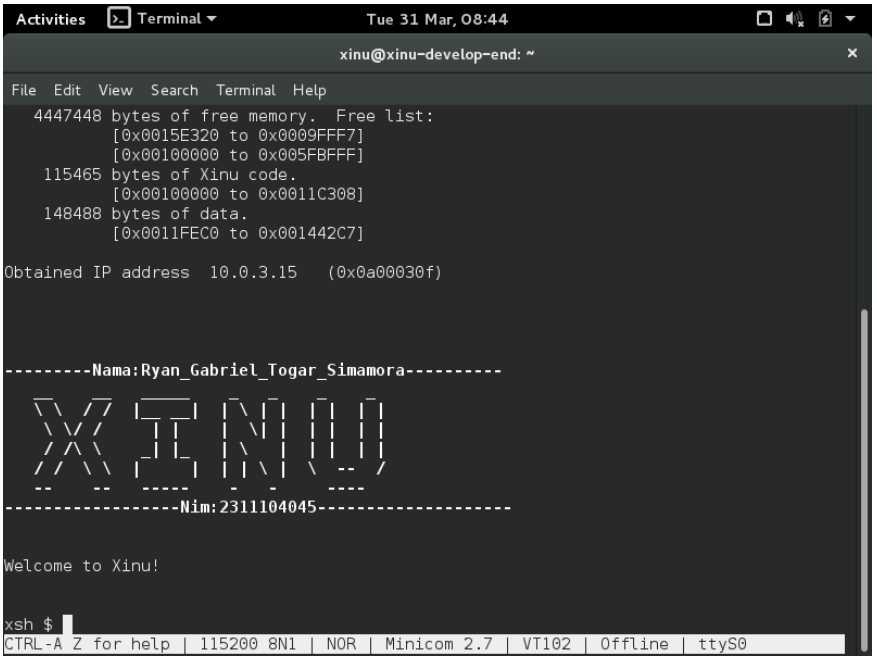
BAB II
HASIL PRAKTIKUM

2.1 Guided Praktikum

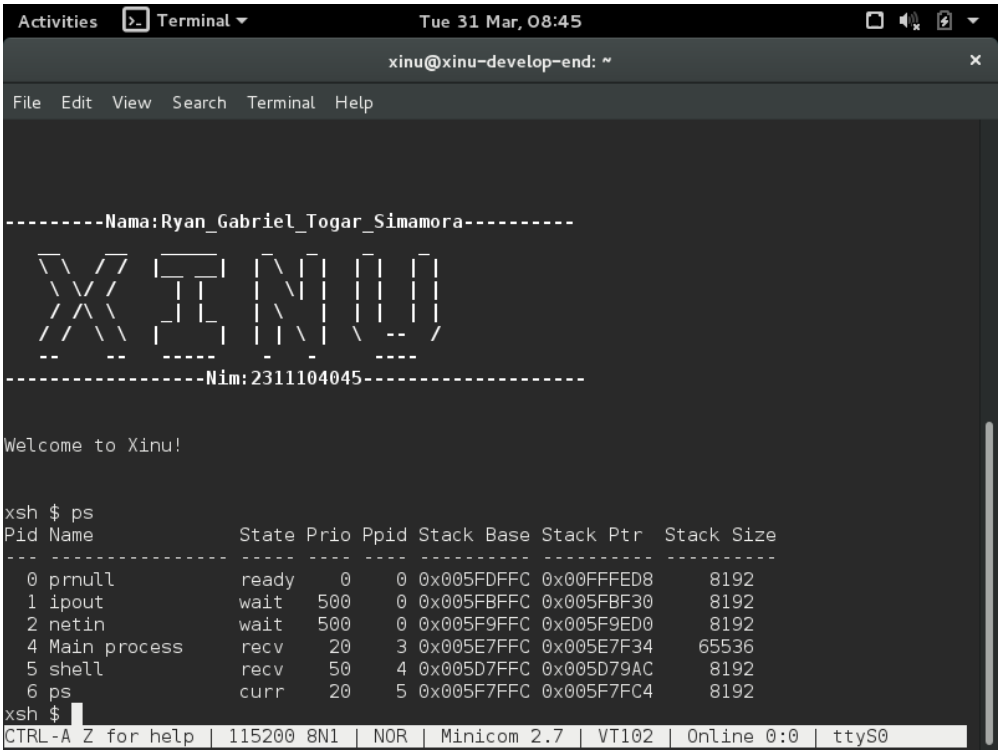
1. Pastikan xinu development dan back end berjalan



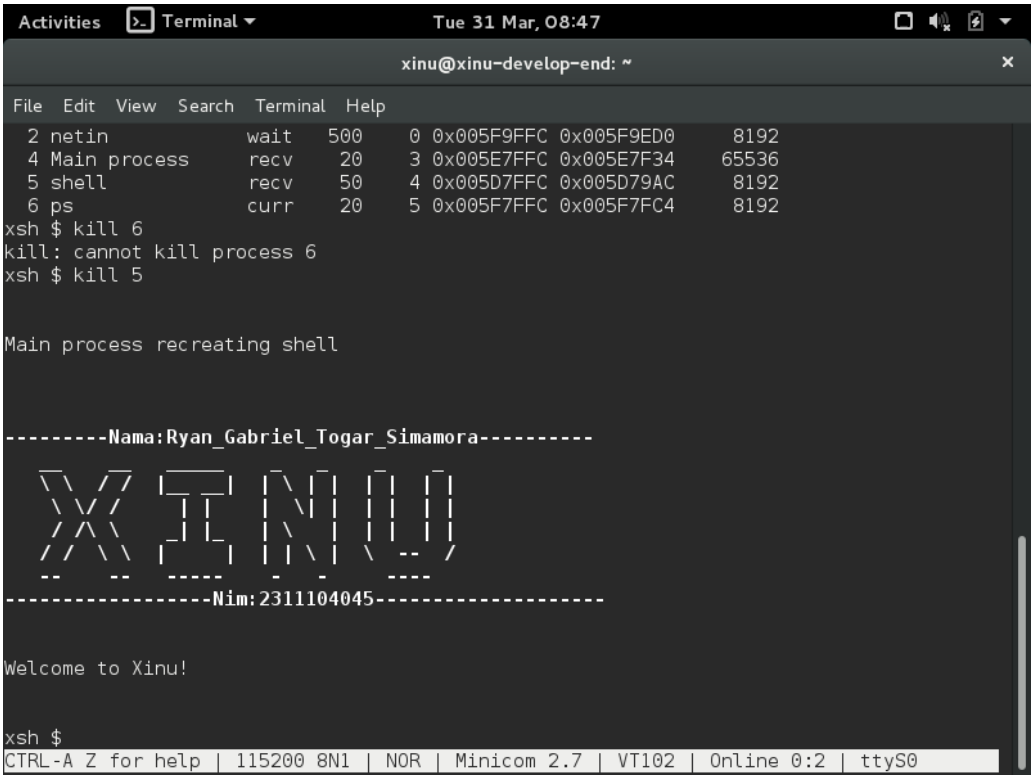
2. Ketikkan sudo minicom lalu enter dan masukkan password xinu



3. Ketikkan ps



4. Kita coba terminate shell dengan perintah kill 5



5. Cek lagi pid processnya denga perintah ps

```
Activities  Terminal  Tue 31 Mar, 08:51
xinu@xinu-develop-end: ~
File  Edit  View  Search  Terminal  Tabs  Help
xinu@xinu-develop-end: ~   xinu@xinu-develop-end: ~
---------Nama:Ryan_Gabriel_Togar_Simamora----------
  __   __  _____   _   _   _   _
  \ \  / /  |_ _|  | \ | |  | |  | |
   \ \/ /    | |   |  \| |  | |  | |
   / /\ \    _| |_  | \   |  | |  | |
  / /  \ \  |     | | | \ |  \  --  /
  --    --  -----   -   -    ----
-----------------Nim:2311104045--------------------

Welcome to Xinu!

xsh $ ps
Pid Name             State Prio Ppid Stack Base Stack Ptr  Stack Size
--- ---------------- ----- ---- ---- ---------- ---------- ----------
  0 prnull           ready    0    0 0x005FDFFC 0x00FFFEB4     8192
  1 ipout            wait   500    0 0x005FBFFC 0x005FBF30     8192
  2 netin            wait   500    0 0x005F9FFC 0x005F9ED0     8192
  4 Main process     recv    20    3 0x005E7FFC 0x005E7F34    65536
  9 shell            recv    20    4 0x005F7FFC 0x005F79AC     4096
 10 ps               curr    20    9 0x005F6FFC 0x005F6DC4     8192
xsh $
```

Terlihat pid process shell berubah dari pid 5 ke pid 9

Beberapa guided selanjutnya udh aku bahas di dasar teori di mulai dari process.h , create.c,kill.c. dan resume .c

Bisa juga ngeliat isi file dari source trail source xinu minggu kemarin

2.2 Jurnal Praktikum

1. [10 Poin] Jawablah pertanyaan berikut ini:

a. Berapa banyaknya maksimum proses yang ada pada Xinu?

Maksimum proses pada Xinu adalah NPROC = **8 proses** (didefinisikan di process.h).

```
Activities  gedit  Tue 31 Mar, 06:32
Open  *process.h  ~/xinu/include  Save
/* process.h - isbadpid */

/* Maximum number of processes in the system */

#ifndef NPROC
#define NPROC          8
#endif
```

b. Berapa maksimal panjang nama suatu proses pada Xinu?

PNMLEN menentukan panjang maksimum nama proses.
Maksimal panjang nama proses adalah **16 karakter**, didefinisikan sebagai:

```
/* Miscellaneous process definitions */

#define PNMLEN          16      /* Length of process "name"            */
#define NULLPROC        0       /* ID of the null process              */
```

c. Berapa nilai prioritas awal pada saat proses dibuat?

Nilai prioritas awal (default priority) saat proses dibuat adalah 20, didefinisikan sebagai:

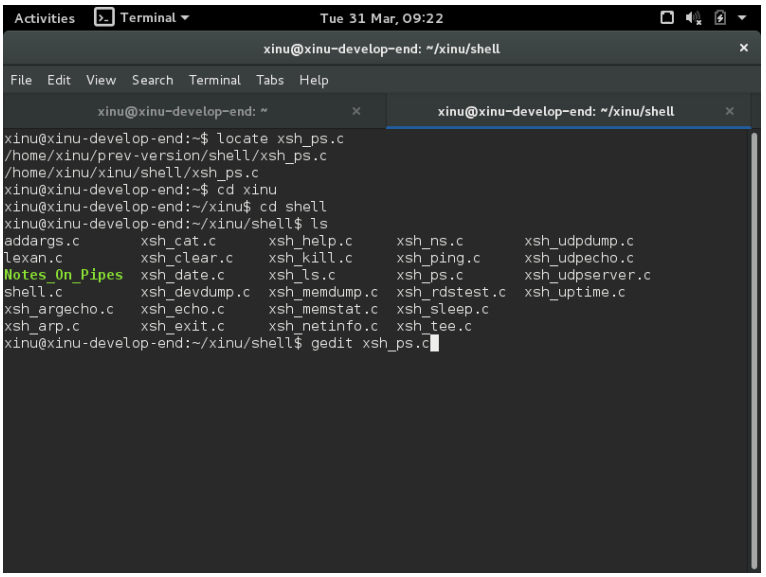
```
#define INITPRIO        20      /* Initial process priority            */
```

d. Ada berapa total state pada Xinu? Sebutkan!

Total terdapat 8 state pada Xinu:

```
/* Process state constants */

#define PR_FREE         0       /* Process table entry is unused       */
#define PR_CURR         1       /* Process is currently running        */
#define PR_READY        2       /* Process is on ready queue           */
#define PR_RECV         3       /* Process waiting for message         */
#define PR_SLEEP        4       /* Process is sleeping                 */
#define PR_SUSP         5       /* Process is suspended                */
#define PR_WAIT         6       /* Process is on semaphore queue       */
#define PR_RECTIM       7       /* Process is receiving with timeout   */
```

2. [20 Poin] Perintah ps adalah perintah untuk menampilkan statistik process yang berjalan. Source code dari ps tersimpan pada file xsh_ps.c. Carilah file tersebut dan beri komentar pada 20 baris terakhir di source code tersebut!



Activities gedit Tue 31 Mar, 10:34

Open xsh_ps.c ~/xinu/shell Save

```c
/* Print header for items from the process table */

        printf("%3s %-16s %5s %4s %4s %10s %-10s %10s %3s %7s\n",
                   "Pid", "Name", "State", "Prio", "Ppid", "Stack Base",
                   "Stack Ptr", "Stack Size" , "Msg", "Content");

        printf("%3s %-16s %5s %4s %4s %10s %-10s %10s %3s %7s\n",
                   "---", "----------------", "-----", "----", "----",
                   "----------", "----------", "----------", "---","-------");

        /* Output information for each process */

        /* Nama : Ryan Gabriel , Nim : 2311104045 */

        for (i = 0; i < NPROC; i++) {
                prptr = &proctab[i];
                if (prptr->prstate == PR_FREE) {  /* Skip unused slots  */
                        continue;
                }
                printf("%3d %-16s %s %4d %4d 0x%08X 0x%08X %8d %3d %7d\n",
                        i, prptr->prname, pstate[(int)prptr->prstate],
                        prptr->prprio, prptr->prparent, prptr->prstkbase,
                        prptr->prstkptr, prptr->prstklen,
                        prptr->prhasmsg,
                        (prptr->prhasmg ? (int)prptr->prmsg: 0));
        }
        return 0;
}
```

C Tab Width: 8 Ln 63, Col 62 INS

3. [35 Poin] Ubahlah perintah ps (source code: xsh_ps.c) pada Xinu sehingga menampilkan informasi tambahan berupa kolom yang berisi total message yang ada pada proses seperti gambar di bawah ini:

```
xsh $ ps
Pid Name             State Prio Ppid Stack Base Stack Ptr  Stack Size Msg Content
--- ---------------- ----- ---- ---- ---------- ---------- ---------- --- -------
  0 prnull           ready    0    0 0x005FDFFC 0x00FFFF04     8192   1       4
  1 rdsproc          susp   200    0 0x005FBFFC 0x005FBFC8    16384   0       0
  2 ipout            wait   500    0 0x005F7FFC 0x005F7F40     8192   0       0
  3 netin            wait   500    0 0x005F5FFC 0x005F5EE0     8192   0       0
  5 Main process     recv    20    4 0x005E3FFC 0x005E3F64    65536   0       0
  6 shell            recv    50    5 0x005D3FFC 0x005D3C7C     8192   0       0
  7 ps               curr    20    6 0x005F3FFC 0x005F3DC8     8192   0       0
```

Kolom Msg adalah banyaknya pesan yang ada dalam proses.
Kolom Content adalah isi dari pesan tersebut.

Langkah pengerjaan:
• Modifikasi source code pada file xsh_ps.c
• Kompilasi ulang Xinu dengan perintah seperti pada modul sebelumnya
• Jalankan Backend VM
• Setelah sistem berjalan, jalankan perintah $ps. Pastikan hasilnya sesuai dengan contoh output pada gambar yang diberikan.
• Screenshot source kode dan output akhir hasil modifikasi

4. [35 Poin] Ubahlah perintah uptime pada Xinu sehingga menampilkan lamanya Xinu sejak booting hanya dalam satuan menit.

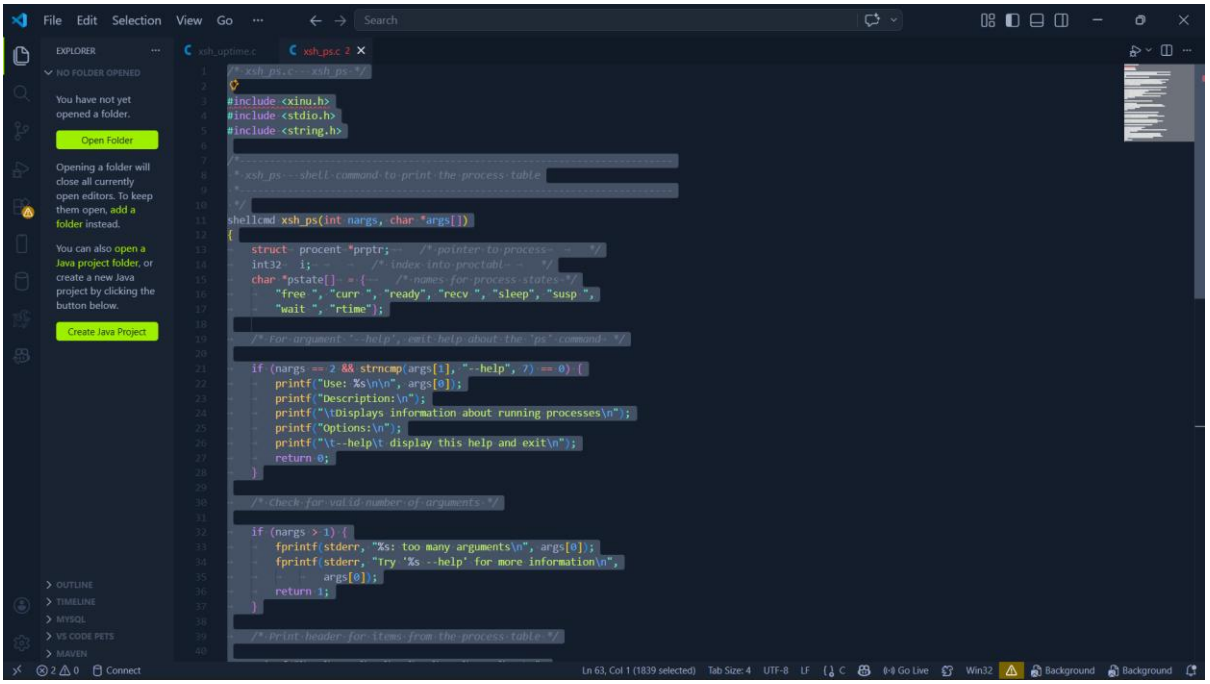
Langkah pengerjaan:
• Modifikasi source code pada file xsh_uptime.c
• Kompilasi ulang Xinu dengan perintah seperti pada modul sebelumnya
• Jalankan Backend VM
• Setelah sistem berjalan, jalankan perintah $uptime. Pastikan hasilnya sesuai dengan contoh output yang diinginkan
• Screenshot source kode dan output akhir hasil modifikasi

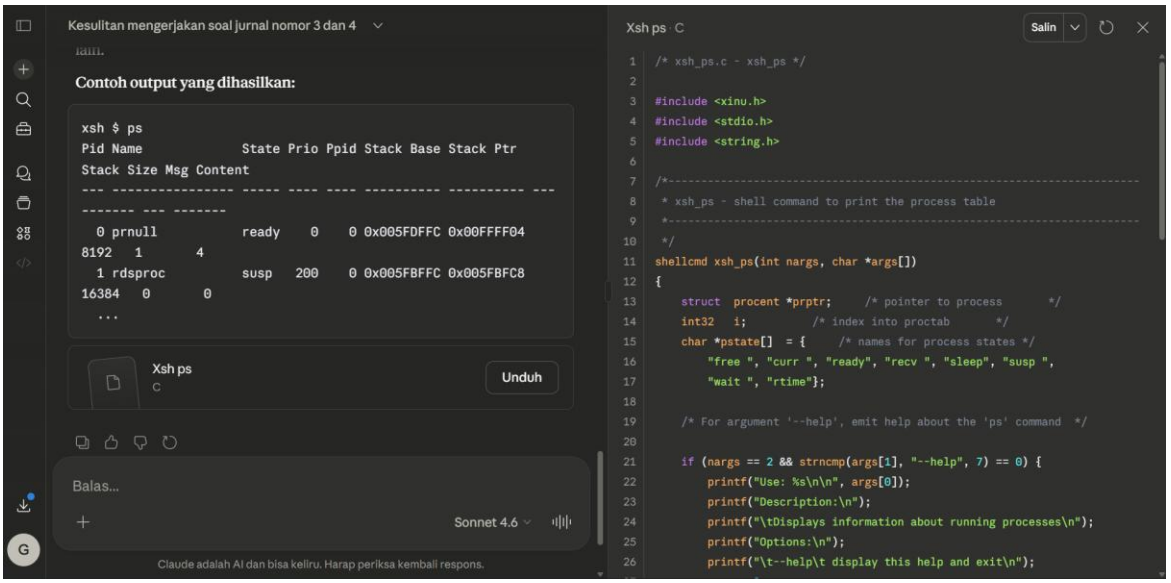
Jawaban No 3 dan 4 :

No.3

Agar mudah dicopas dan modifikasi jadi saya langsung copy dari folder source code xinu yang pernah dikirim dari grup dan saya buka vs code dan saya suruh ai claude saya yang gacor untuk ubah codingan ini agar sesuai dengan yang diminta pada soal

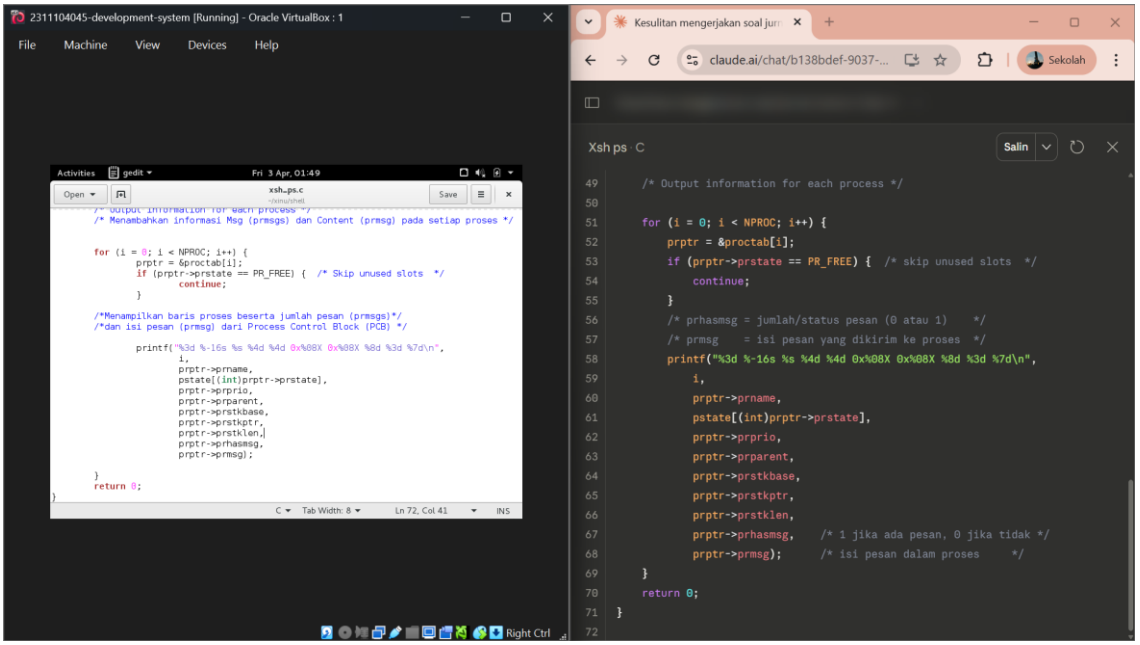
File xsh_ps.c





Nanti File yang di hasil kan ai akan saya masukkan ke file xinu nya langsung dengan perintah gedit untuk mengedit file dan melakukan perubahan kode menyesuaikan hasil ai , menurut saya sepertinya cara ini akan berhasil , letss gooo mencoba

Disini saya udh menyesuaikan source codenya



Jalankan Perintah make di xinu/include

```
Activities  Terminal  Fri 3 Apr, 00:53
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Tabs  Help
xinu@xinu-develop-end: ~/xinu/compile    xinu@xinu-develop-end: ~
xinu@xinu-develop-end:~/xinu/compile$ make
Rebuilding the .defs file
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

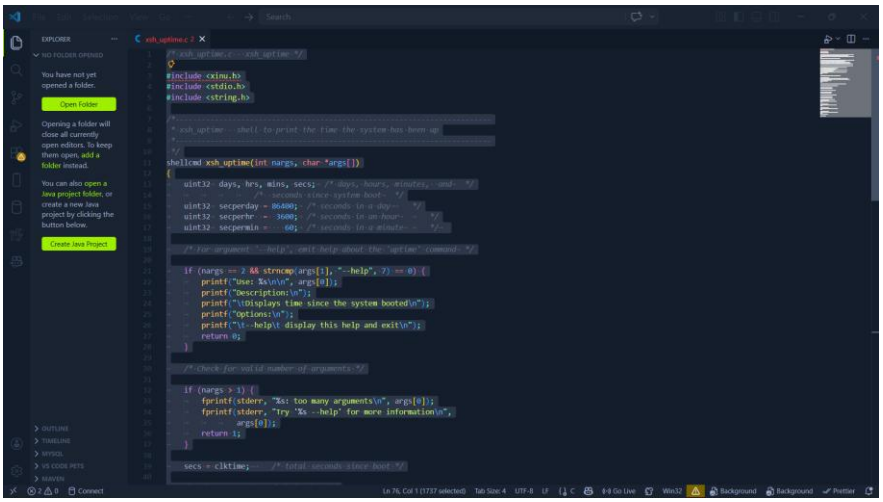
Hasil output yang dimodifikasi

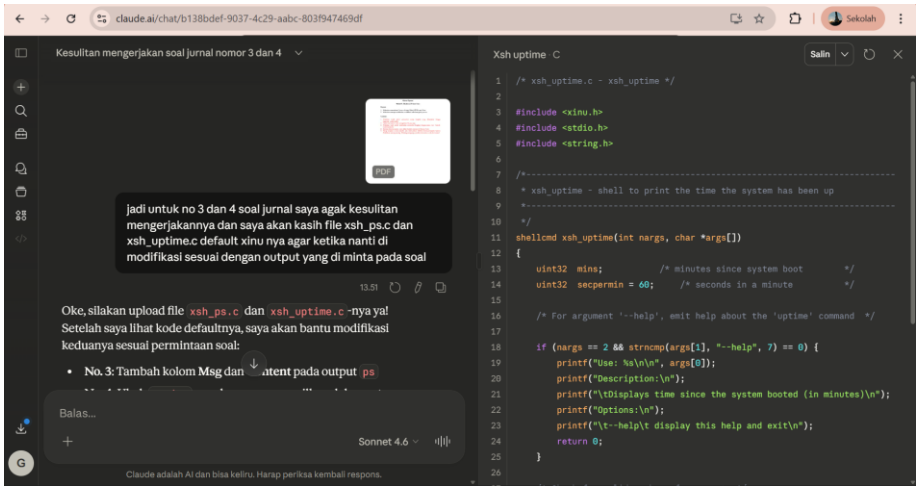
```
Activities  Terminal  Fri 3 Apr, 01:40
xinu@xinu-develop-end: ~
File  Edit  View  Search  Terminal  Tabs  Help
xinu@xinu-develop-end: ~/xinu/compile    xinu@xinu-develop-end: ~
  __   __   _____   _   _   _   _
  \ \  / /  |__ __|  | \ | |  | |  | |
   \ \/ /     | |    |  \| |  | |  | |
   / /\ \    _| |_   | \   |  | |  | |
  / /  \ \  |     |  | | \ |  \  --  /
  --    --  -----    -   -    ----
------------------Nim:2311104045--------------------

Welcome to Xinu!

xsh $ uptime
Xinu has been up 0 minute(s)
 xsh $ ps
Pid Name             State Prio Ppid Stack Base Stack Ptr  Stack Size Msg Content
--- ---------------- ----- ---- ---- ---------- ----------         --- ------- Content
  0 prnull           ready    0    0 0x005FDFFC 0x00FFFEE4     8192   1       3
  1 ipout            wait   500    0 0x005FBFFC 0x005FBF30     8192   0       0
  2 netin            wait   500    0 0x005F9FFC 0x005F9ED0     8192   0       0
  4 Main process     recv    20    3 0x005E7FFC 0x005E7F34    65536   0       0
  5 shell            recv    50    4 0x005F7FFC 0x005F79AC     8192   0       6
  7 ps               curr    20    5 0x005F5FFC 0x005F5FC4     8192   0       0
xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:35 | ttyS0
```

NO.4

Agar mudah copas dan modifikasi jadi saya langsung copy dari folder source code xinu yang pernah dikirim dari grup dan saya buka vs code dan saya suruh ai claude saya yang gacor untuk ubah codingan ini agar sesuai dengan yang diminta pada soal

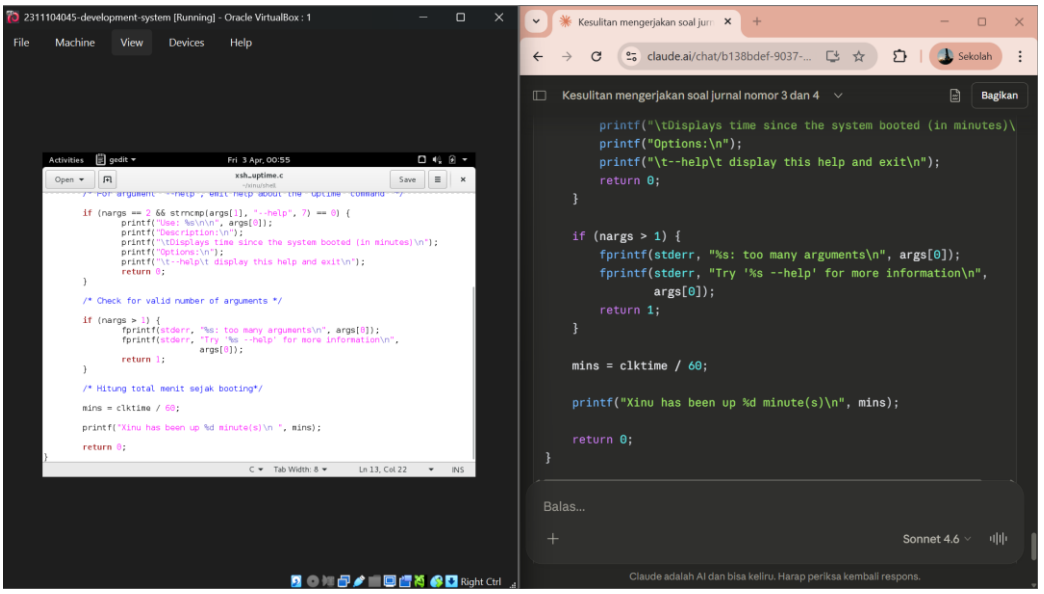
File xsh_uptime.c



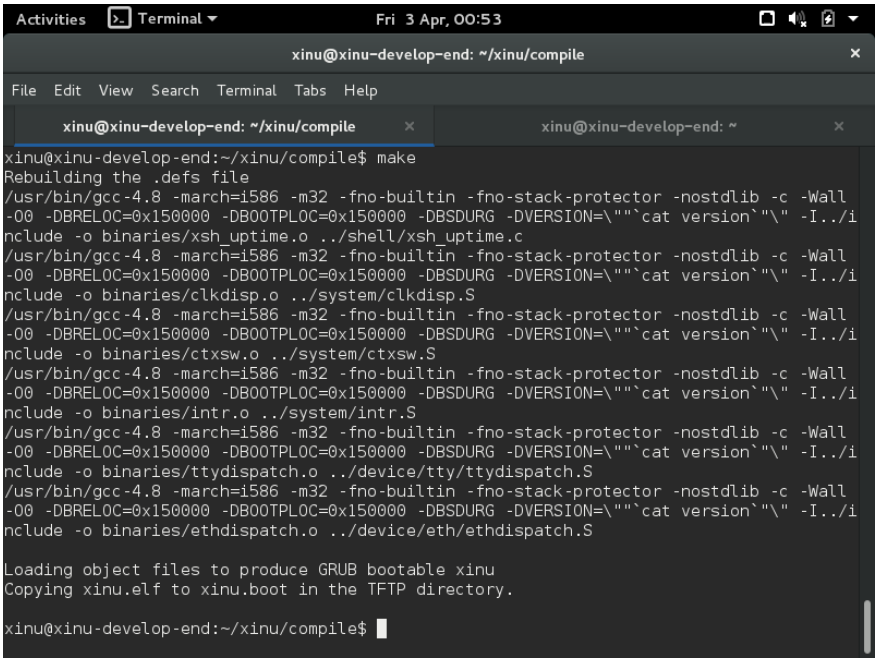


Nanti File yang di hasil kan ai akan saya masukkan ke file xinu nya langsung dengan perintah gedit untuk mengedit file dan melakukan perubahan kode menyesuaikan hasil ai , menurut saya sepertinya cara ini akan berhasil , letss gooo mencoba

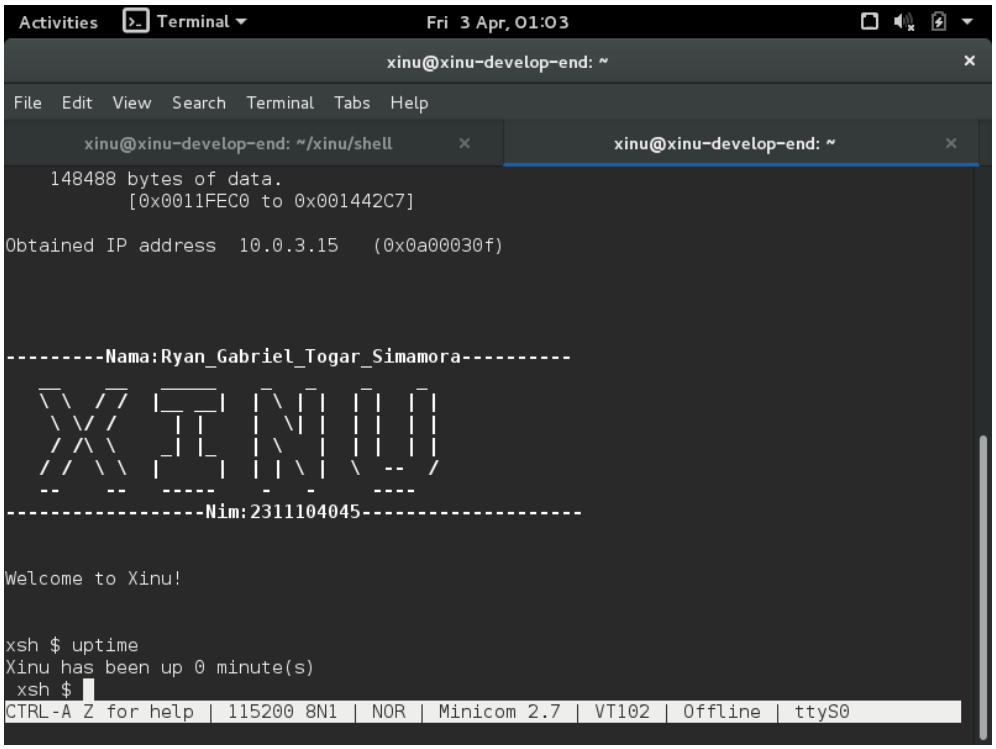
Disini saya udh menyesuaikan source codenya



Jalankan Perintah make di xinu/include



Hasil output yang dimodifikasi



BAB III
PENUTUP

3.1 Kesimpulan

Modul ini memberikan pemahaman mendalam tentang bagaimana sistem operasi Xinu mengelola proses di dalam kernelnya. Melalui eksplorasi berbagai file seperti process.h, create.c, kill.c, dan resume.c, serta pengujian langsung melalui Minicom, kita dapat melihat bagaimana proses diciptakan, dijalankan, disuspensi, hingga diakhiri secara sistematis.

Beberapa poin penting yang dapat disimpulkan:

1. File process.h merupakan fondasi utama dalam manajemen proses Xinu. Di dalamnya terdapat definisi penting seperti:
   - Jumlah maksimum proses (NPROC)
   - Status proses (PR_READY, PR_SLEEP, PR_SUSP, dll.)
   - Struktur procent yang menyimpan semua informasi tiap proses dalam tabel proctab[].

1. Fungsi create() pada create.c bertugas membuat proses baru, mengalokasikan memori stack, menetapkan prioritas, serta mengisi tabel proses agar proses siap dijalankan scheduler.
2. Fungsi kill() pada kill.c digunakan untuk menghapus proses dari sistem. Fungsi ini menutup resource seperti deskriptor file, membebaskan memori stack, dan memperbarui status proses menjadi PR_FREE, memastikan sistem tetap efisien tanpa resource leak.
3. Fungsi resume() pada resume.c memungkinkan proses yang disuspend untuk diaktifkan kembali. Proses berpindah dari status PR_SUSP ke PR_READY, siap untuk dijadwalkan kembali oleh scheduler.
4. Melalui pengujian di Minicom, diperlihatkan bahwa kernel Xinu dapat dijalankan secara stabil dengan shell interaktif. Perintah seperti help dan ps menunjukkan daftar perintah dan proses aktif, membuktikan bahwa sistem manajemen proses bekerja dengan baik.

   Secara keseluruhan, eksplorasi proses di Xinu menggambarkan prinsip dasar manajemen proses dalam sistem operasi. Mulai dari pembuatan, eksekusi, penjadwalan, penundaan, hingga penghentian proses. Dengan arsitektur yang sederhana dan modular, Xinu menjadi platform ideal untuk memahami konsep fundamental sistem operasi seperti proses, scheduler, interrupt, dan resource management secara langsung dan praktis.

   File process.h berfungsi sebagai pusat pengaturan dan definisi proses di sistem Xinu. Mulai dari batas jumlah proses, status proses, hingga parameter inisialisasi proses — semuanya diatur di sini untuk mendukung manajemen proses yang efisien.